

## Solution Requirements Specification

|                      |                 |                       |                                     |
|----------------------|-----------------|-----------------------|-------------------------------------|
| <b>Date:</b>         | 3 October 2026  | <b>Team ID:</b>       | UNM1611SHARVESHARAM-SWTID-2026-8110 |
| <b>Project Name:</b> | AI BlogNest API | <b>Maximum Marks:</b> | 4 Marks                             |

### Functional and Non-Functional Requirements:

This document defines the comprehensive operational specifications for AI BlogNest API. Functional Requirements (FR) define what the system does, while Non-Functional Requirements (NFR) specify quality, performance, and security criteria.

#### Step 1: Functional Requirements (FR) — Part 1

| S.No | FR Category                    | Requirement Description                                                                                                                                                                                 | Priority |
|------|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| 1    | <b>Authentication</b>          | Users must securely register with name, unique email, and minimum 6-character password. Passwords are encrypted with bcrypt salt factor 10. Login issues a signed JWT token valid for 7 days.           | High     |
| 2    | <b>Authorization Levels</b>    | Public users can read published blogs (GET /api/blogs, GET /api/blogs/:id). Authenticated users can create blogs and invoke AI generation. Only the original author can edit or delete their own posts. | High     |
| 3    | <b>External Interfaces</b>     | Integration with Google Gemini Generative AI API via @google/generative-ai SDK. Communicates with MongoDB Atlas cloud database via Mongoose ODM over TLS connection string.                             | High     |
| 4    | <b>Transactions Processing</b> | Handles atomic CRUD database transactions: user document creation, blog document creation, idempotent query retrieval, atomic update, and deletion with cascading checks.                               | High     |

## Functional Requirements (Part 2) & Non-Functional Requirements

### Step 1: Functional Requirements (FR) — Part 2

| S.No | FR Category               | Requirement Description                                                                                                                                                                                                                                | Priority |
|------|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| 5    | Reporting & Querying      | Endpoint GET /api/blogs returns list of blogs sorted descending by creation date, dynamically populating author name and email fields from the User collection.                                                                                        | Medium   |
| 6    | Business Rules & AI Logic | <ul style="list-style-type: none"> <li>Generates structured 100-150 word blogs on given topic.</li> <li>Regex engine sanitizes raw markdown/asterisks.</li> <li>Auto-infers category across 9 domains (Tech, Health, etc.) if not provided.</li> </ul> | High     |
| 7    | Compliance & Privacy      | Credentials never stored in plaintext; API secrets stored in .env and ignored by git; strict request payload validation via express-validator to prevent injection attacks.                                                                            | High     |
| 8    | AI Summarization          | Endpoint POST /api/ai/summarize accepts arbitrary blog content and generates a single crisp executive summary paragraph using Gemini AI.                                                                                                               | Medium   |

### Step 2: Non-Functional Requirements (NFR) — Part 1

| S.No | NFR Category        | Requirement Description                                              | Target Metric / Acceptance Criteria                                     |
|------|---------------------|----------------------------------------------------------------------|-------------------------------------------------------------------------|
| 1    | Performance & Speed | Fast response times for API queries and external LLM dispatch.       | CRUD latency < 100 ms;<br>Gemini generation < 2.5 seconds               |
| 2    | Scalability         | Stateless architecture capable of handling concurrent user requests. | Supports 500+ concurrent connections;<br>horizontal scaling ready       |
| 3    | Security & Privacy  | Cryptographic password hashing and tamper-evident JWT authorization. | Bcrypt salt=10; HMAC-SHA256 tokens;<br>OWASP-compliant sanitized inputs |

## Non-Functional Requirements (Part 2) & Traceability Matrix

### Step 2: Non-Functional Requirements (NFR) — Part 2

| S.No | NFR Category                         | Requirement Description                                                                                                 | Target Metric / Acceptance Criteria                                               |
|------|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| 4    | <b>Reliability &amp; Uptime</b>      | Robust server uptime with automatic MongoDB reconnection and centralized error recovery.                                | 99.9% availability; zero uncaught exceptions crashing the Node process            |
| 5    | <b>Usability &amp; API Standards</b> | Strict RESTful conventions with semantic HTTP status codes (200, 201, 400, 401, 403, 404, 422, 500) and JSON responses. | 100% compliant with REST standards; Thunder Client / Postman collections provided |
| 6    | <b>Maintainability</b>               | Clean MVC architecture separating routes, controllers, models, and services for straightforward maintenance.            | Modular codebase; zero circular dependencies; Prettier formatting                 |

### Requirements Traceability & Verification Matrix:

| Req ID | Feature Description          | Source File / Implementation          | Verification Method                 | Status   |
|--------|------------------------------|---------------------------------------|-------------------------------------|----------|
| FR-01  | User Registration & Bcrypt   | authController.js / User.js           | Thunder Client: POST /register      | Verified |
| FR-02  | JWT Login & Bearer Token     | authController.js / authMiddleware.js | Thunder Client: POST /login         | Verified |
| FR-03  | Blog CRUD Operations         | blogController.js / Blog.js           | Thunder Client: /api/blogs suite    | Verified |
| FR-04  | Gemini Blog Post Generation  | aiController.js / geminiService.js    | Thunder Client: POST /generate-blog | Verified |
| FR-05  | Gemini Blog Summarization    | aiController.js / geminiService.js    | Thunder Client: POST /summarize     | Verified |
| FR-06  | Category Auto-Classification | aiController.js (inferCategory)       | Unit test across 9 keyword sets     | Verified |